


**UFC**

**UNIVERSIDADE FEDERAL DO CEARÁ**
**CAMPUS ITAPAJÉ**

# Práticas de Verilog

**Autor: Pablo Caio Felismino dos Santos**

**Disciplina: Sistemas Embarcados**

Itapajé, Outubro, 2026

# 1 Ambiente utilizado

Os códigos foram escritos em Verilog no VS Code e simulados no DigitalJS, que gera o circuito a partir do código e mostra a forma de onda dos sinais. Nos circuitos, o bloco `$procdff` é um flip-flop D e o `$procmux` é um multiplexador. Nas formas de onda, verde é nível 1 e vermelho é nível 0.

# 2 Prática 01 - Portas lógicas

Foram feitos quatro módulos separados: AND, OR, NOT e XOR. Todos têm o `clk` e a saída `Q`. A AND, a OR e a XOR têm as entradas `A` e `B`, e a NOT tem só a entrada `A`. A operação é feita dentro de um `always @(posedge clk)`, então o resultado passa por um flip-flop e `Q` só muda na borda de subida do clock.

## 2.1 Porta AND

A saída é 1 só quando `A` e `B` são 1.

Listing 1: and_porta.v

```
1  module and_porta (
2      input wire A,
3      input wire B,
4      input wire clk,
5      output reg Q
6  );
7
8  always @(posedge clk) begin
9      Q <= A & B;
10 end
11
12 endmodule
```

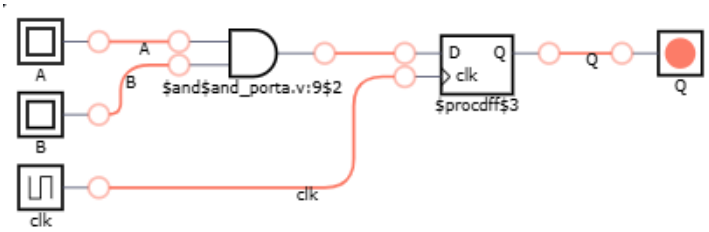



Figura 1: Circuito da porta AND.

Na forma de onda, `A` sobe primeiro, mas `Q` continua em 0 porque `B` ainda está em 0. Quando `B` também vai a 1, `Q` sobe na borda de clock seguinte. Quando `A` volta a 0, `Q` só cai na próxima borda do clock.

## 2.2 Porta OR

A saída é 1 quando pelo menos uma das entradas é 1.

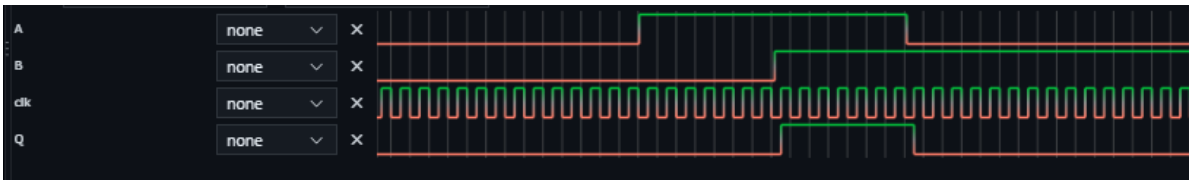



Figura 2: Forma de onda da porta AND.

Listing 2: or_porta.v

```
module or_porta (
    input wire A,
    input wire B,
    input wire clk,
    output reg Q
);

always @(posedge clk) begin
    Q <= A | B;
end

endmodule
```

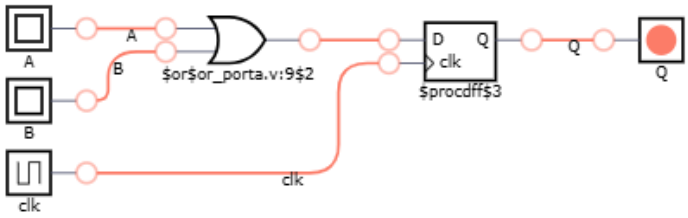



Figura 3: Circuito da porta OR.

Com `A` e `B` em 0, `Q` fica em 0. Quando `A` vai a 1, `Q` sobe na borda seguinte do clock. Depois `B` também sobe e, quando `A` cai, `Q` continua em 1, porque ainda há uma entrada em 1.

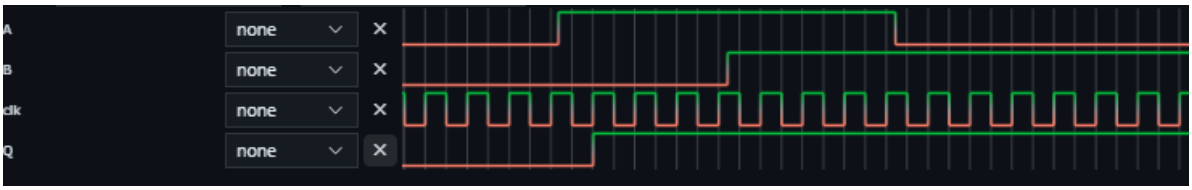



Figura 4: Forma de onda da porta OR.

## 2.3 Porta NOT

A saída é o inverso da entrada. Em Verilog usa o operador `~`. Como as outras portas, a NOT também passa por um flip-flop com `clk`.

Listing 3: not_porta.v

```
module not_porta (
    input wire A,
    input wire clk,
    output reg Q
);

always @(posedge clk) begin
    Q <= ~A;
end

endmodule
```




Figura 5: Circuito da porta NOT.

Com `A` em 0, `Q` fica em 1. Quando `A` vai a 1, `Q` cai para 0 só na borda de subida seguinte do clock.

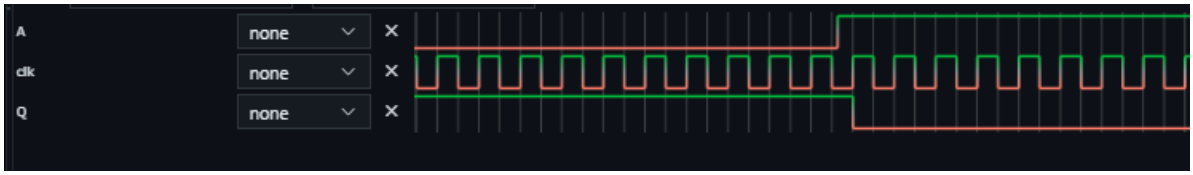



Figura 6: Forma de onda da porta NOT.

## 2.4 Porta XOR

A saída é 1 quando as entradas são diferentes.

Listing 4: xor_porta.v

```
module xor_porta (
    input wire A,
    input wire B,
    input wire clk,
    output reg Q
);

always @(posedge clk) begin
    Q <= A ^ B;
end

endmodule
```

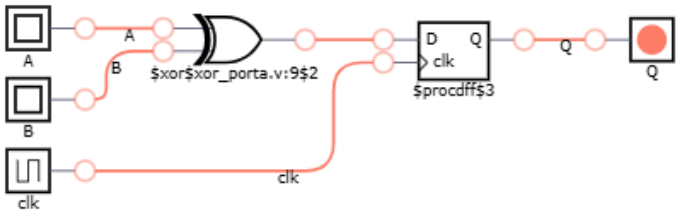



Figura 7: Circuito da porta XOR.

No começo `Q` ainda está indefinido (cinza) e passa a 0 na primeira borda do clock. A saída vai a 1 quando as entradas ficam diferentes (10 ou 01) e volta a 0 quando as duas ficam em 1. Em todos os casos a mudança só aparece em `Q` na borda seguinte do clock.

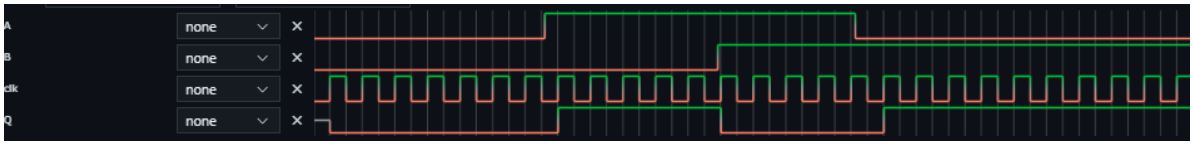



Figura 8: Forma de onda da porta XOR.

## 3 Prática 02 - Multiplexador 2:1

O multiplexador escolhe qual entrada vai para a saída de acordo com `S`: com `S = 0` a saída `Y` recebe `A`, e com `S = 1` recebe `B`. Foi usado o operador ternário com `assign`. Como o enunciado pede só as entradas `A`, `B` e `S` e a saída `Y`, esse módulo é combinacional: não tem clock nem flip-flop.

Listing 5: mux_porta.v

```
1  module mux_porta (
2      input wire A,
3      input wire B,
4      input wire S,
5      output wire Y
6  );
7
8  assign Y = S ? B : A;
9
10 endmodule
```

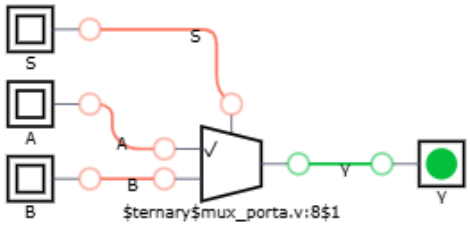



Figura 9: Circuito do MUX 2:1.

Na forma de onda, com `S = 1`, mudar `A` não altera `Y`, que só sobe quando `B` vai a 1. Como não há flip-flop, `Y` muda no mesmo instante em que a entrada escolhida muda, sem esperar o clock.




Figura 10: Forma de onda do MUX 2:1.

## 4 Prática 03 - Registrador de 1 bit (Flip-flop D)

É um flip-flop D sozinho: o valor de `D` é copiado para `Q` na borda de subida do clock. Fora da borda, `Q` mantém o último valor.

Listing 6: registrador_porta.v

```
module registrador_porta (
    input wire D,
    input wire clk,
    output reg Q
);

always @(posedge clk) begin
    Q <= D;
end

endmodule
```

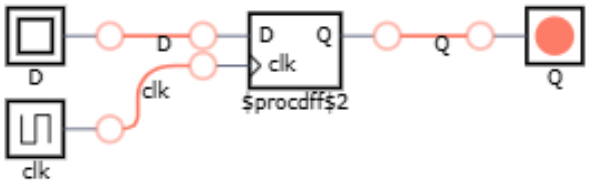



Figura 11: Circuito do registrador de 1 bit.

Na forma de onda, quando `D` sobe, `Q` só sobe na próxima borda de subida do clock, e o mesmo acontece quando `D` desce.

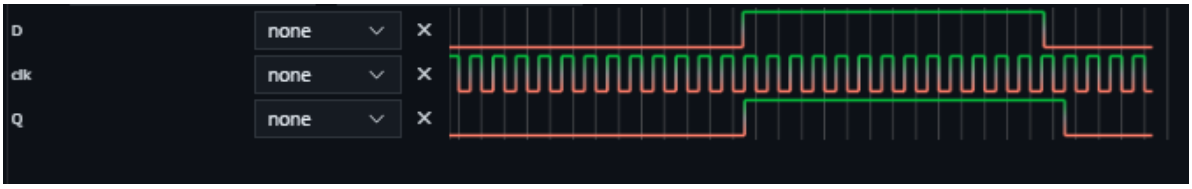



Figura 12: Forma de onda do registrador de 1 bit.

## 5 Prática 04 - FSM: Torneira Automática

Máquina de estados que controla uma torneira com sensor de presença. As entradas são `clk`, `rst` (reset), `sensor` (mãos embaixo da torneira) e `timer_fim` (tempo máximo acabou). A saída `valvula` abre (1) ou fecha (0) a água. Os estados são `OFF` (00), `DETECCAO` (01), `LIGADA` (10) e `DESLIGANDO` (11).

O código tem três blocos `always`: um guarda o estado atual, outro calcula o próximo estado com um `case` e o último liga a válvula só no estado `LIGADA`.

Listing 7: torneira_fsm.v

```
module torneira_fsm (
    input wire clk,
    input wire rst,
    input wire sensor,
    input wire timer_fim,
```

```
6       output reg valvula
7   );
8
9   localparam OFF      = 2'b00;
10  localparam DETECCAO = 2'b01;
11  localparam LIGADA   = 2'b10;
12  localparam DESLIGANDO = 2'b11;
13
14  reg [1:0] estado_atual, proximo_estado;
15
16  // Registrador de estado
17  always @(posedge clk or posedge rst) begin
18      if (rst)
19          estado_atual <= OFF;
20      else
21          estado_atual <= proximo_estado;
22  end
23
24  // Logica do proximo estado
25  always @(*) begin
26      proximo_estado = estado_atual;
27
28      case (estado_atual)
29          OFF: begin
30              if (sensor)
31                  proximo_estado = DETECCAO;
32          end
33
34          DETECCAO: begin
35              if (sensor)
36                  proximo_estado = LIGADA;
37              else
38                  proximo_estado = OFF;
39          end
40
41          LIGADA: begin
42              if (!sensor || timer_fim)
43                  proximo_estado = DESLIGANDO;
44          end
45
46          DESLIGANDO: begin
47              proximo_estado = OFF;
48          end
49
50          default: proximo_estado = OFF;
51      endcase
52  end
53
54  // Saida
55  always @(*) begin
56      valvula = (estado_atual == LIGADA);
57  end
58
59  endmodule
```

Funcionamento:

- `OFF`: torneira fechada. Se o sensor detectar as mãos, vai para `DETECCAO`;
- `DETECCAO`: se o sensor continuar ativo no próximo ciclo, vai para `LIGADA`; se não, volta para `OFF`;
- `LIGADA`: válvula aberta. Sai para `DESLIGANDO` quando as mãos são retiradas ou o tempo acaba;
- `DESLIGANDO`: válvula fechada, volta para `OFF` no próximo ciclo.

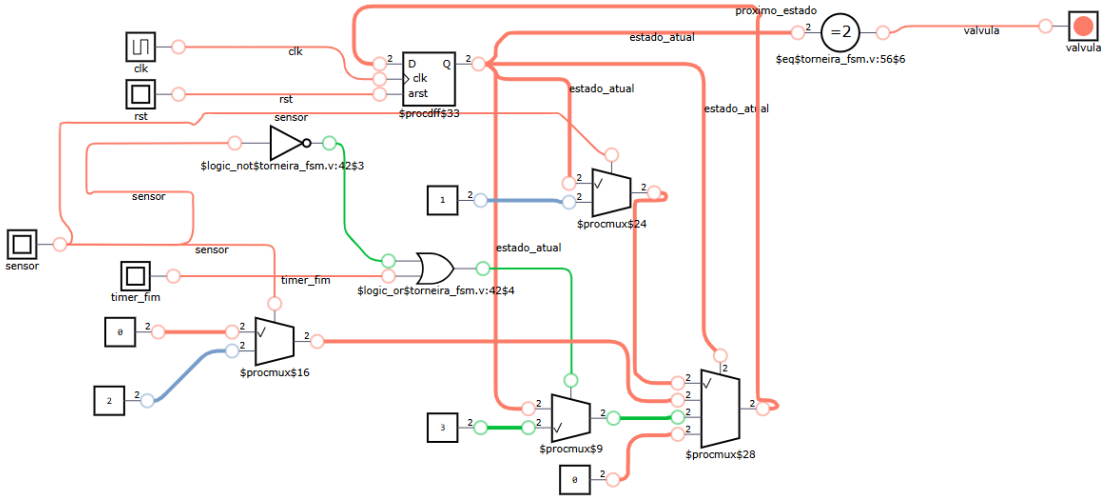



Figura 13: Circuito da FSM da torneira automática.

No circuito, o `$procdff$33` é o registrador do estado (com o `arst` ligado no reset), os `$procmux` calculam o próximo estado e o `$eq` compara o estado com 2 (`LIGADA`) para ligar a válvula.

Na forma de onda, os estados aparecem como números (0 = `OFF`, 1 = `DETECCAO`, 2 = `LIGADA`, 3 = `DESLIGANDO`). Com o `rst` em 1 no começo, o estado fica em 0. Depois que o `sensor` sobe, o estado passa por 1 e 2 e a válvula abre ao chegar em 2. Quando `timer_fim` vai a 1 com o sensor ainda ativo, o estado vai para 3 e volta para 0, e a válvula fecha; como o sensor continua ativo, a máquina reinicia o ciclo e a válvula abre só por pulsos curtos enquanto `timer_fim` fica em 1. Quando `timer_fim` volta a 0, o estado fica em 2 e a válvula abre. Por fim, quando o sensor vai a 0, o estado vai para 3, volta para `OFF` e a válvula fecha. O `proximo_estado` aparece sempre um ciclo adiantado em relação ao `estado_atual`.

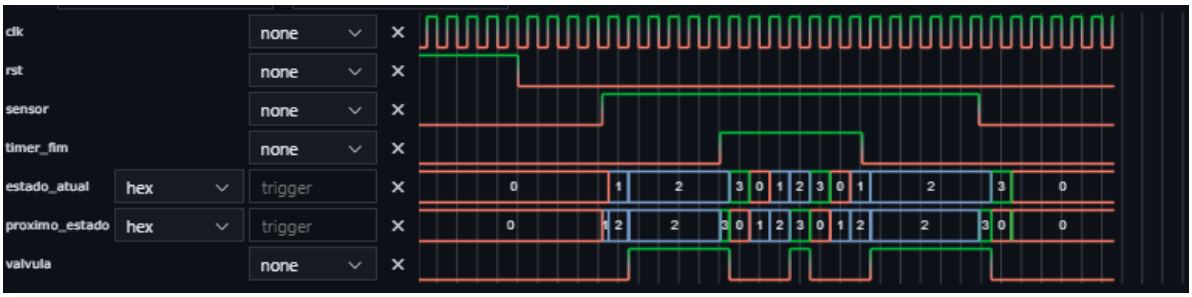



Figura 14: Forma de onda da FSM da torneira automática.

# 6 Conclusão

Todas as práticas funcionaram como esperado nas simulações. Nas portas e no registrador, a saída `Q` só muda na borda de subida do clock. No MUX, que é combinacional, a saída `Y` muda assim que a entrada escolhida muda. Na FSM, a torneira passou pelos estados na ordem certa e a válvula só abriu no estado `LIGADA`.